

SEARCH TYPEAHEAD SYSTEM

Project Report

Property	Value
Project	Pulse Search
Dataset	Wikimedia Pageviews — 120,000 queries
Implementation	Node.js, Express, SQLite, consistent hashing
Report date	June 22, 2026

A low-latency search suggestion application with cache-aside reads, recency-aware ranking, consistent-hash distribution, and batched count updates.

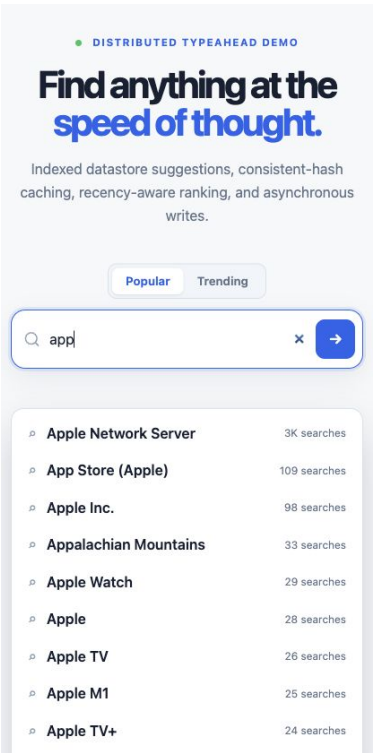


Table of Contents

1. Executive Summary
2. Architecture Diagram and Explanation
3. Dataset Source and Loading Instructions
4. API Documentation
5. Design Choices and Trade-offs
6. Performance Report
7. Setup, Testing, and Conclusion
8. References

1. Executive Summary

Pulse Search is a complete search typeahead application that returns up to ten prefix-matching suggestions while a user types. Suggestions can be ranked by historical popularity or by a recency-aware trending score. SQLite is the reliable primary store, three independent Redis cache nodes are routed through consistent hashing, and an asynchronous batch writer reduces database pressure.

Area	Implementation
Frontend	Responsive HTML/CSS/JavaScript; 300 ms debounce
API	Node.js and Express
Primary store	SQLite via better-sqlite3
Cache	3 Redis processes; 450 virtual ring positions
Deployment	Docker Compose app + Redis ports 6379/6380/6381
Trending	10 × 30-second buckets with exponential decay
Batch writes	5 seconds or 50 unique buffered queries
Dataset	120,000 Wikimedia page titles and counts

2. Architecture Diagram and Explanation

Search Typeahead System Architecture

Cache-aside reads, asynchronous batched writes, and recency-aware ranking

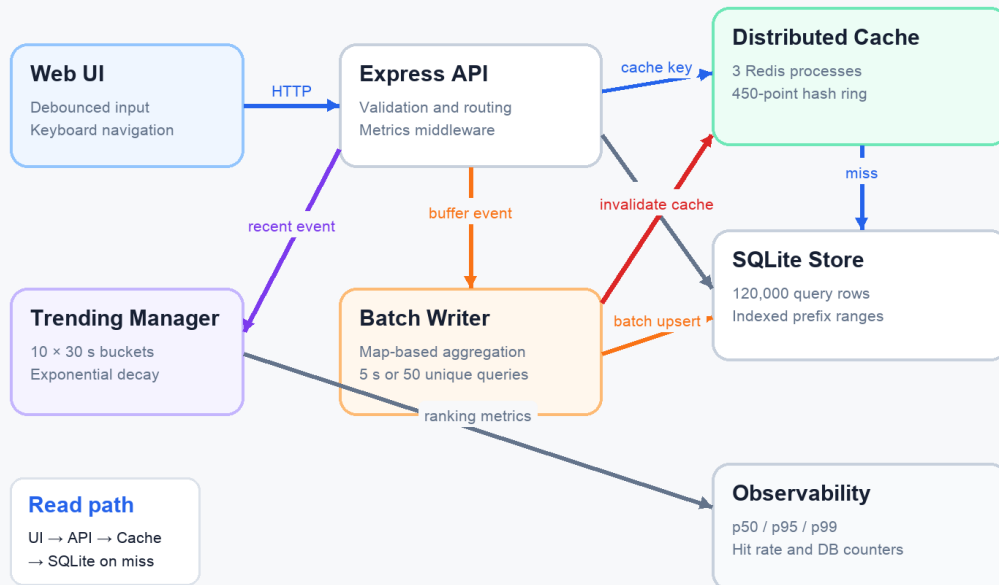


Figure 1. Read path, write path, and observability flow.

2.1 Read Path

- The browser sends a debounced GET /suggest request.
- The API normalizes the prefix and builds a mode-specific cache key.
- Consistent hashing selects one cache node; a hit returns the top ten immediately.
- A miss queries SQLite through an indexed prefix range, sorts by count, and caches the result for 60 seconds.

2.2 Write Path

POST /search records recent activity and aggregates the normalized query in an in-memory map. At five seconds or 50 unique queries, one SQLite transaction upserts the batch and affected prefix cache keys are invalidated. Graceful shutdown flushes the remaining buffer.

2.3 Data Model

```
CREATE TABLE queries (  
  normalized_query TEXT PRIMARY KEY,  
  display_query TEXT NOT NULL,  
  count INTEGER NOT NULL DEFAULT 0,  
  updated_at INTEGER NOT NULL  
);
```

3. Dataset Source and Loading Instructions

The project uses Wikimedia Pageviews. Page titles become queries and real page views provide popularity counts.

Property	Value
Dataset	Wikimedia Pageviews
Source	pageviews-20250101-000000.gz
Period	January 1, 2025, 00:00 UTC

Property	Value
Filter	English Wikipedia desktop (en)
License	CC0 1.0
Final size	120,000 unique query,count rows

3.1 Transformation

- Stream the gzip source and retain English Wikipedia rows.
- Decode page titles, replace underscores, and exclude administrative namespaces.
- Normalize and deduplicate titles, then retain the 120,000 highest-count entries.
- Write dataset/queries.csv using the required query,count schema.

3.2 Loading

```
npm install
npm run setup
npm start
```

The setup command reuses the included CSV when metadata matches the pinned source and loads SQLite in 10,000-row transactions.

3.3 Redis and Docker Startup

```
docker compose up --build

# Redis containers only + local app
npm run redis:up
CACHE_BACKEND=redis npm start
```

Compose starts three independent Redis processes on host ports 6379, 6380, and 6381. The application uses REDIS_URLS and its own consistent-hashing router rather than Redis Cluster slot routing.

4. API Documentation

Method	Endpoint	Purpose
GET	/suggest?q=&mode=	Prefix suggestions
POST	/search	Queue a search count update
GET	/trending	Current top ten trending queries
GET	/cache/debug?prefix=	Cache owner and hit state
GET	/stats	Performance and system counters
POST	/flush	Development-only manual flush
POST	/cache/nodes/:id	Development membership simulation

4.1 GET /suggest

```
GET /suggest?q=app&mode=count
```

```
{
  "suggestions": [{"query": "Apple Network Server", "count": 2956}],
  "cached": false,
  "latencyMs": 0.143
}
```

q is required, limited to 200 characters, and normalized for mixed-case matching. Empty input returns an empty array. mode is count or trend.

4.2 POST /search

```
POST /search
{"query": "Apple"}

202 Accepted
{"message": "Searched", "query": "Apple"}
```

The response confirms acceptance. Persistence is eventual because the count is written during the next batch flush.

5. Design Choices and Trade-offs

Decision	Reason	Trade-off
SQLite	Durable, transactional, zero-ops	Single-node storage
Prefix ranges	Indexed primary-store fallback	Cold result sorting
Three Redis nodes	Shared independent cache processes	One-host Compose lacks zone redundancy
Virtual nodes	Even distribution, limited remapping	More ring metadata
TTL + invalidation	Freshness with fast reads	O(query length) invalidation
Batch buffer	Aggregates repeated writes	Hard-crash loss risk
5-minute trend window	Fast and demonstrable decay	Resets on restart
Vanilla UI	No build step	Less component reuse

5.1 Trending Formula

```
weightedRecentCount =  $\Sigma(\text{bucketCount} \times 0.9^{\text{bucketAge}})$ 
historical =  $\log_{1p}(\text{totalCount}) / \log_{1p}(\text{maxCount})$ 
recent = weightedRecentCount /  $\max(1, \text{maxWeightedRecentCount})$ 
trendScore =  $0.4 \times \text{historical} + 0.6 \times \text{recent}$ 
```

Ten rotating buckets track five minutes of activity. Decay and expiration prevent a short-lived spike from remaining permanently over-ranked.

5.2 Failure Trade-off

SIGINT and SIGTERM trigger a final flush. A hard crash may lose up to one interval of buffered counts. Production mitigation would use Kafka, Redis Streams, or a disk-backed WAL.

6. Performance Report

Measurement	Result
Dataset rows	120,000
Cold HTTP p95	2.875 ms
Warm HTTP p50	0.203 ms
Warm HTTP p95	0.369 ms
Warm HTTP p99	0.506 ms
Server suggestion p95	0.070 ms
Cache hit rate	95.24%
Database reads	12
Batch transactions	1
Rows updated	2

Measurement	Result
Events flushed	100
Write reduction	50:1

6.1 Method and Interpretation

The benchmark uses a temporary database copy, sends cold and warm requests over ten prefixes, submits 100 repeated search events, flushes, and reads /stats. A 95.24% hit rate confirms effective caching. One hundred events became two row updates in one transaction, demonstrating a 50:1 reduction. Ring ownership ranged from 23.83% to 26.99%. Recent activity moved Apple above Apple Network Server in trend mode.

6.2 Limitations

This benchmark uses the memory backend for reproducibility. The Docker Redis deployment adds network serialization but still shares one host, so results demonstrate relative behavior rather than production capacity.

7. Setup, Testing, and Conclusion

```
npm install
npm run setup
npm test
npm start
npm run benchmark
```

Ten automated tests cover ingestion, prefix ordering, case normalization, edge cases, TTL and invalidation, consistent hashing, batching, unseen query insertion, trending decay, APIs, and metrics. The system satisfies the full assignment rubric while remaining easy to run and explain during a viva.

8. References

- Wikimedia Pageviews: <https://dumps.wikimedia.org/other/pageviews/readme.html>
- Dataset file: <https://dumps.wikimedia.org/other/pageviews/2025/2025-01/pageviews-20250101-000000.gz>
- CC0 1.0: <https://creativecommons.org/publicdomain/zero/1.0/>
- Internal documentation: README.md, docs/architecture.md, docs/performance-report.md